



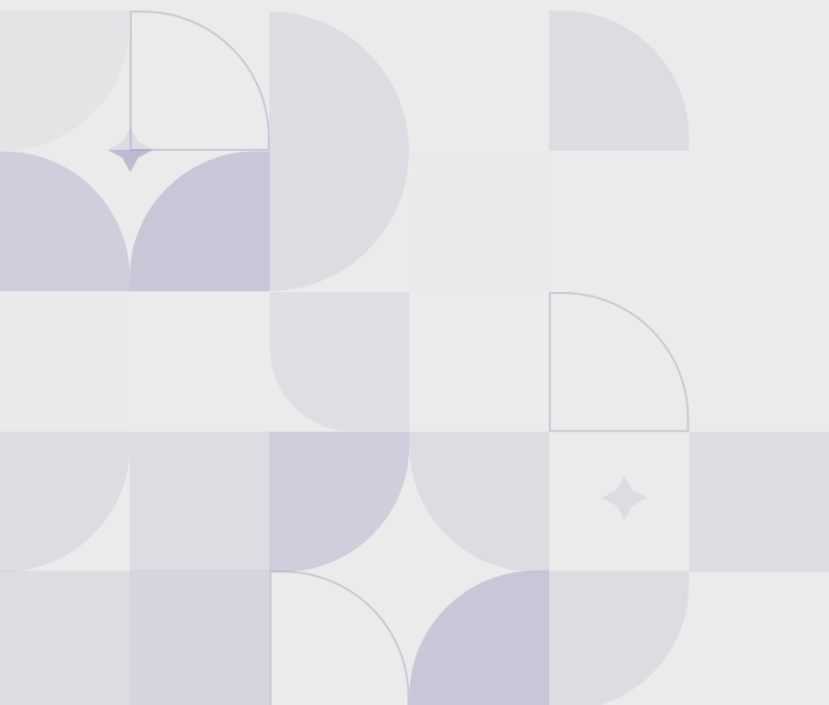
Модуль 9. Основи продуктивності та оптимізації

Урок 2. Кешування та оптимізація роботи з базою даних

Реальні важелі швидкості: кеш і БД

Мета уроку

1. розібрати типи кешування і коли вони доречні
2. навчитися аналізувати й прискорювати запити до БД
3. показати, як боротися з N+1 через eager loading (Laravel/Symfony)
4. зрозуміти роль індексів і пагінації в продуктивності API



Кеш - не plaster для поганої архітектури

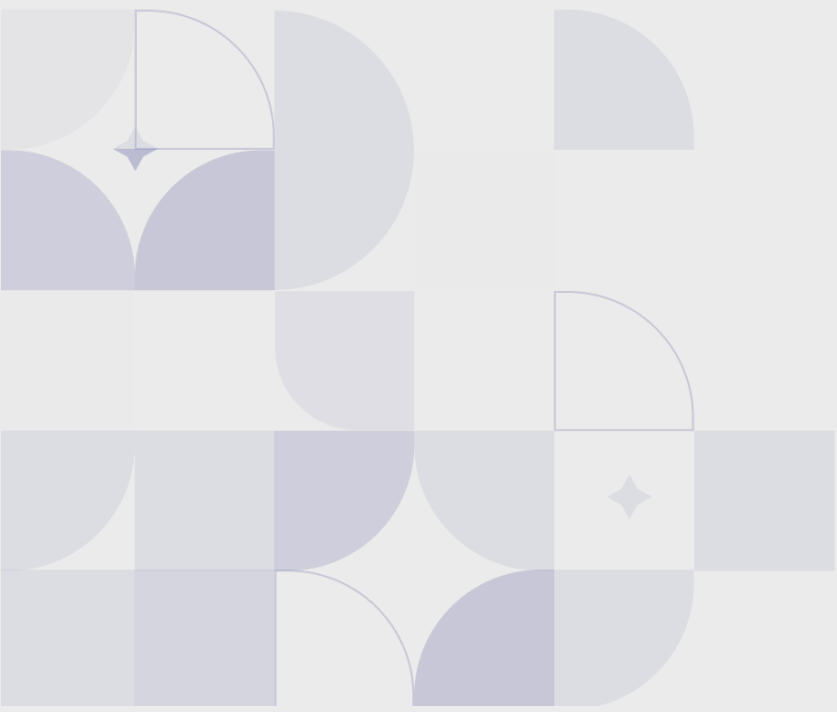
Проблеми:

- неправильне кешування → застарілі дані, дивні баги
- кеш не виправляє N+1 чи відсутність індексів
- найскладніше - інвалідація (коли очищати/оновлювати)

Що ми можемо кешувати

Високорівнево:

- дані (довідники, налаштування)
- результати запитів (важкі агрегації, фільтри)
- об'єкти (моделі/DTO)
- відповіді API (read-only, публічні дані)



Довідники - перший кандидат на кеш

Приклади:

- валюти
- типи операцій
- налаштування комісій/лімітів (якщо рідко змінюються)
- коди помилок шлюзу

Приклади:

- кешуємо результат (**Currency::all()**), задаємо TTL, інвалідуємо при оновленні

Практика: CurrencyService з кешем і invalidate()

Кешуємо довідник, очищаємо при оновленні

Зберігаємо важкі агрегації

Приклади:

- сумарні платежі за період
- складні фільтри з JOIN'ами
- статистика по рахунках

Принцип:

- ключ включає параметри (account_id, період)
- TTL або інвалідація при створенні/оновленні платежів

Коли можна кешувати цілий JSON

Підходить:

- GET, read-only, публічні/рідко змінювані дані (тарифи, умови, каталог)

Не підходить:

- персоналізовані / real-time дані без спецлогіки
- POST/PATCH/DELETE



Кешувати теж треба з головою

Не кешуємо:

- дуже часті зміни без продуманної інвалідації
- баланс/ліміти для критичних операцій
- будь-що, де застарілі дані = серйозна фінансова помилка

Redis: швидке in-memory сховище

- in-memory key-value, TTL для ключів
- у Laravel - стандартний драйвер кешу
- у docker-стеку вже використовується для черг (Модуль 5)

Як можна нашкодити кешем

Приклади:

- відсутність TTL → переповнення пам'яті
- кешування всього підряд (без інвалідації)
- великі JSON як значення (повільна серіалізація)
- відсутність fallback до БД під час збоїв Redis

Спочатку дивимось на SQL, а не на PHP

Інструменти:

- Laravel Telescope/Debugbar
- Doctrine Profiler, Blackfire (Symfony)
- логування SQL в dev

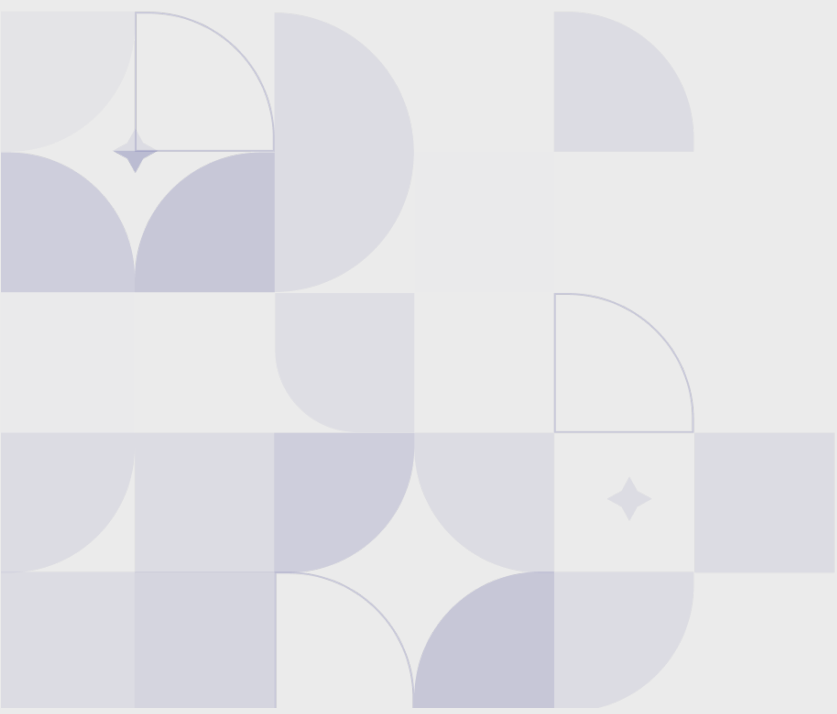
1 запит на список + N запитів на зв'язки

Наслідки:

- лінійне зростання часу й навантаження
- ризик таймаутів за великих N

Рішення:

- eager loading (**with()** / JOIN'и)



with() замість lazy-завантаження в циклі

- **Payment::with(['account', 'user'])->where(...)->paginate(...)**
- кількість запитів фіксована (1–2), а не зростає з N

JOIN замість лінивого доступу в колекції

- DQL/QueryBuilder із join'ами та **addSelect()** для зв'язків
- Doctrine Paginator для списків

Практика: PaymentRepository без N+1

Списки платежів з account/user за 1–2 запити

Індекси пришвидшують читання, але уповільнюють запис ✨

Ключові моменти:

- індексувати поля WHERE/JOIN/ORDER BY (account_id, created_at, status)
- індексувати зовнішні ключі (foreign keys)
- кожен індекс має мати «юзкейс»

Без пагінації список не масштабується

- обмежує розмір відповіді
- стабілізує час і навантаження
- must-have для будь-яких потенційно великих списків (платежі, транзакції)

Важкі операції — у чергу, не в HTTP

Приклади:

- генерація звіту за платежами
- експорт виписки
- важкі агрегації

Як можна “оптимізувати” й усе зіпсувати

Приклади:

- кешування без аналізу й правил інвалідації
- ігнорування N+1
- відсутність індексів на великих таблицях
- «мікрооптимізації» PHP-коду замість одного slow-query

Де у вас списки й як вони влаштовані?

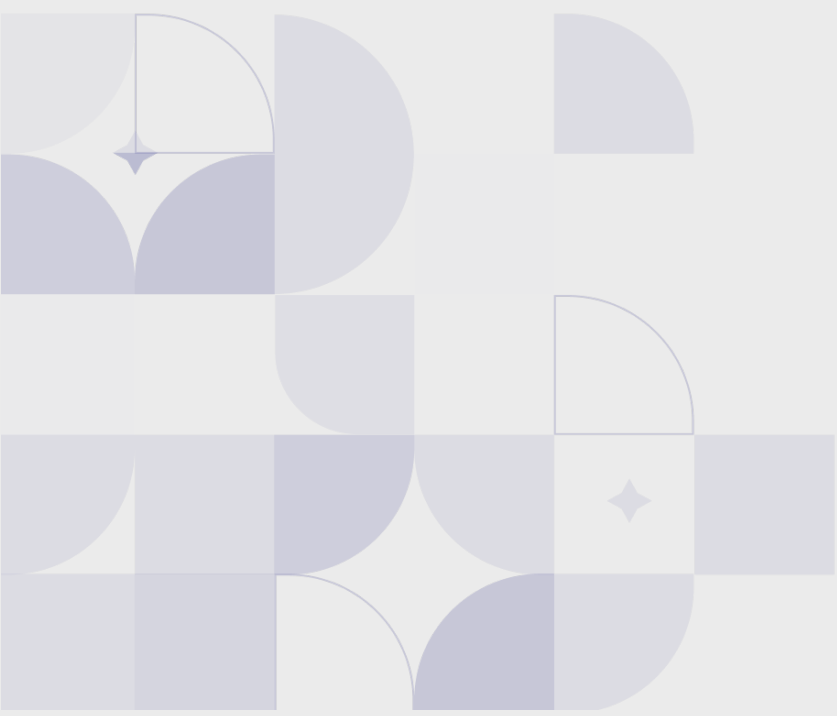
Завдання:

- обрати кілька endpoint'ів (платежі, рахунки, виписки)
- позначити: чи є пагінація, чи можна кешувати, з яким TTL

Рахуємо запити й думаємо про eager loading/ індекси

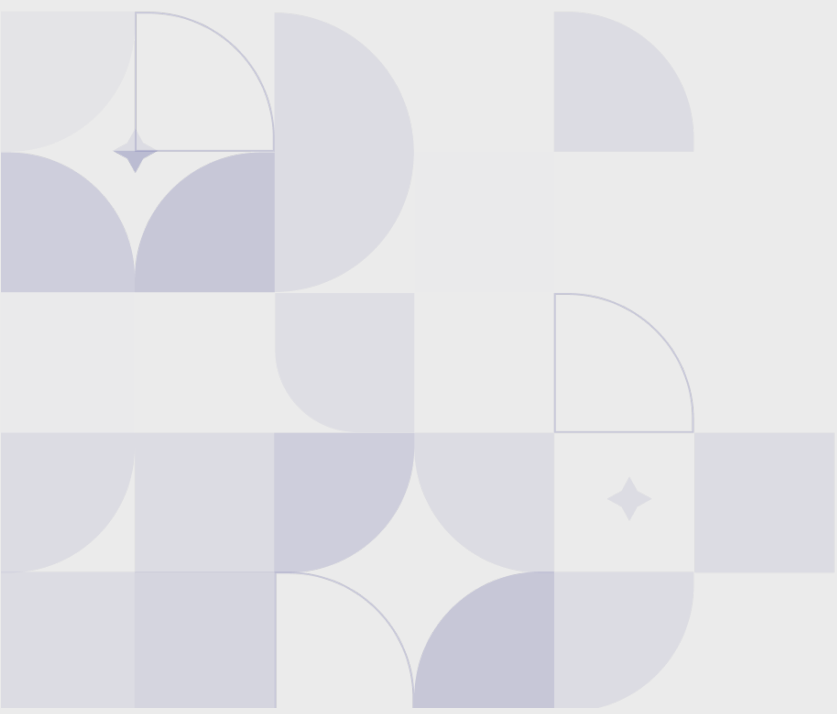
Завдання:

- за допомогою Telescope/Debugbar/профайлера подивитися: кількість запитів, час
- записати, де потенційно N+1, де потрібні індекси



Що ми сьогодні конкретизували

- типи кешування та їх застосування; роль Redis і ризики його неправильного використання
- профілювання SQL, N+1, eager loading
- індекси, пагінація, jobs як ключові інструменти



Головні важелі швидкості — БД і кеш

В більшості веб-застосунків найбільший виграш у продуктивності дає не “магія з кодом”, а чітко налаштовані запити до БД, кеш і пагінація

Дякуємо за увагу, питання?